

TRƯỜNG ĐẠI HỌC KỸ THUẬT CÔNG NGHIỆP
KHOA ĐIỆN TỬ



BÀI TIỂU LUẬN
MÔN HỌC

ĐẢM BẢO CHẤT LƯỢNG & KIỂM THỬ PHẦN
MỀM

CONTEXT-DRIVEN TESTING TRONG DỰ ÁN PHẦN MỀM

HỌ VÀ TÊN SINH VIÊN : Nguyễn Tuấn Anh
MSSV : K225480106002
LỚP : K58KTP.01
GVHD : TS.NGUYỄN TUẤN LINH

THÁI NGUYÊN - 2026

BÀI TẬP LỚN

MÔN HỌC: ĐẢM BẢO CHẤT LƯỢNG & KIỂM THỬ PHẦN
MỀM

BỘ MÔN : CÔNG NGHỆ THÔNG TIN

Sinh viên: Nguyễn Tuấn Anh MSSV : K225480106002

Lớp: K58KTP.01

Ngành: Kỹ Thuật Máy Tính

Giáo viên hướng dẫn: TS. Nguyễn Tuấn Linh

Ngày giao đề : 03/06/2026..... Ngày hoàn thành : 21/06/2026

Tên đề tài : Context-Driven Testing trong dự án phần mềm

1. Nội Dung Báo Cáo :

- *Thống kê và phân tích*
- *Thực hành*

2. Sản phẩm :

- *Slide thuyết trình*
- *File báo cáo tiểu luận*
- *Link Github : https://github.com/nguynta04-sys/BTL_DBCL-KTPM-NguyenTuanAnh*

GIÁO VIÊN HƯỚNG DẪN

(Ký và ghi rõ họ tên)

NHẬN XÉT CỦA GIÁO VIÊN

.....

.....

.....

.....

.....

.....

.....

.....

Thái Nguyên, Ngày...Tháng...Năm 2026
GIÁO VIÊN HƯỚNG DẪN
(Kí và ghi rõ họ tên)

MỤC LỤC

DANH MỤC BẢNG VÀ HÌNH VẼ.....	6
LỜI NÓI ĐẦU	7
CHƯƠNG 1: MỞ ĐẦU	8
1.1. Lý do chọn đề tài	8
1.2. Mục tiêu nghiên cứu	9
1.3. Phạm vi nghiên cứu	10
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ TỔNG QUAN.....	11
2.1. Phân tích các mô hình phân phối phần mềm và sự chuyển dịch của hoạt động kiểm thử	11
2.1.1. Mô hình Thác nước (Waterfall) và V-Model truyền thống.....	11
2.1.2. Mô hình Agile (Scrum và Extreme Programming - XP)	11
2.1.3. Mô hình DevOps và Chuyển giao liên tục (Continuous Delivery)	12
2.2. Bản chất và 7 nguyên tắc cốt lõi của Trường phái Kiểm thử định hướng bối cảnh (CDT).....	12
2.3. Nghiên cứu sâu về các kỹ thuật cụ thể trong Chương 10.....	15
2.3.1. Khung kiểm thử tinh gọn (Lean Test Canvas).....	15
2.3.2. Sơ đồ tổng điều tra rủi ro (Census of Risk) thay thế cho Test Case.....	16
2.3.3. Phương pháp tiếp cận trực quan hóa qua Bản đồ tư duy (Mindmap Regression)	16
CHƯƠNG 3: VẬN DỤNG VÀ THỰC HÀNH TẠI NGŨ CẢNH CỤ THỂ	17
3.1. Mô tả chi tiết bài toán và ngữ cảnh thực tế của hệ thống Quản lý PC	17
3.1.1. Đặt vấn đề và kiến trúc hệ thống.....	17
3.1.2. Phân tích bối cảnh dự án (Project Context Analysis) theo mô hình CDT	17
3.2. Hiện thực hóa giải pháp kiểm thử CDT tinh gọn trên Visual Studio Code (VS Code)	18
3.3. Phân tích kết quả, thảo luận và đánh giá hiệu quả toàn diện	20
3.3.1. Đánh giá định lượng về mặt thời gian và chi phí vận hành.....	20
3.3.2. Bảng tổng hợp so sánh các chỉ số chất lượng cốt lõi.....	21

CHƯƠNG 4: KẾT LUẬN VÀ KIẾN NGHỊ	23
4.1. Tổng kết nội dung và các phát hiện chính	23
4.2. Hạn chế mang tính khách quan của giải pháp.....	24
4.3. Định hướng mở rộng và Kiến nghị trong tương lai.....	24
TÀI LIỆU THAM KHẢO	26
SLIDE.PPTX	27

DANH MỤC BẢNG VÀ HÌNH VẼ

Hình 2.2 : Bảng nguyên tắc cốt lõi của CDT

Hình 3.2 : Sơ đồ rủi ro

Hình 3.3.2 Hình so sánh các chỉ số chất lượng

LỜI NÓI ĐẦU

Trong kỷ nguyên công nghệ số hiện nay, sự phát triển mạnh mẽ của các hệ thống phần mềm đòi hỏi quy trình sản xuất không chỉ cần nhanh chóng mà còn phải đảm bảo chất lượng tối ưu trước khi tới tay người dùng. Chính vì vậy, kiểm thử phần mềm đã trở thành một giai đoạn then chốt, không thể tách rời trong chu trình phát triển hệ sinh thái ứng dụng. Tuy nhiên, đối với các dự án phần mềm quy mô nhỏ – nơi nguồn lực nhân sự khan hiếm, ngân sách hạn chế và yêu cầu từ khách hàng liên tục thay đổi – việc áp dụng một quy trình kiểm thử truyền thống (Scripted Testing) với hệ thống tài liệu Test Case công kênh thường bộc lộ nhiều bất cập, gây lãng phí thời gian và làm giảm tính linh hoạt của đội ngũ phát triển.

Nhận thức được thách thức đó, em đã lựa chọn đề tài: "Nghiên cứu và áp dụng Kiểm thử định hướng bối cảnh (Context-Driven Testing) trong dự án phần mềm nhỏ" cho bài tập lớn môn học Đảm bảo chất lượng và Kiểm thử phần mềm. Phương pháp kiểm thử định hướng bối cảnh (CDT) không coi trọng các quy tắc cố định, mà tập trung vào việc tìm kiếm giải pháp tối ưu nhất, phù hợp nhất với thực trạng của dự án thông qua việc tùy biến chiến lược linh hoạt dựa trên bốn yếu tố cốt lõi: Rủi ro, Ngân sách, Đội ngũ nhân sự và Mức độ biến động của yêu cầu.

Trong phạm vi bài báo cáo này, em xin trình bày từ cơ sở lý thuyết nền tảng của CDT cho đến quy trình triển khai thực hành cụ thể trên môi trường tích hợp Visual Studio Code (thông qua sơ đồ tư duy kịch bản và công cụ kiểm thử API Thunder Client). Qua đó, đề tài minh chứng cho tính hiệu quả, tiết kiệm chi phí và khả năng thích ứng cao của phương pháp này đối với các dự án phần mềm quy mô nhỏ hiện nay.

CHƯƠNG 1: MỞ ĐẦU

1.1. Lý do chọn đề tài

Trong kỷ nguyên số hóa và sự bùng nổ của các hệ thống thông tin, quy trình sản xuất phần mềm đã và đang trải qua những bước chuyển dịch mang tính bước ngoặt. Từ các mô hình mang tính tuần tự, đóng băng yêu cầu như Thác nước (Waterfall), ngành công nghiệp phần mềm hiện đại đã tiến hóa mạnh mẽ sang các mô hình phân phối liên tục (Continuous Delivery) và linh hoạt (Agile). Sự dịch chuyển này không chỉ thay đổi cách thức các kỹ sư lập trình viết mã nguồn, mà còn tái định hình hoàn toàn bản chất và vai trò của công tác đảm bảo chất lượng phần mềm (QA/Testing). Theo Heusser và Larsen (2024), việc thay đổi mô hình phân phối sẽ kéo theo sự thay đổi bắt buộc trong chiến lược kiểm thử, bởi các rủi ro kỹ thuật không còn xuất hiện tập trung ở cuối chu kỳ mà phân tán liên tục trong từng vòng lặp phát triển.

Đối với các tổ chức công nghệ có quy mô lớn, việc duy trì một phòng ban QA độc lập với hàng chục kỹ sư kiểm thử cùng hệ thống hạ tầng giả lập tự động công kênh là điều khả thi. Tuy nhiên, đối với các dự án phần mềm quy mô nhỏ (Small-scale software projects), bài toán đảm bảo chất lượng lại vấp phải những rào cản vô cùng khắc nghiệt. Những dự án này thường đặc trưng bởi: ngân sách cực kỳ giới hạn, thời gian bàn giao (Time-to-market) bị siết chặt bởi áp lực cạnh tranh, và đặc biệt là sự khan hiếm nhân sự chuyên trách khi một lập trình viên thường phải đảm nhận nhiều vai trò cùng lúc.

Nếu áp dụng một cách máy móc các quy trình kiểm thử truyền thống (Scripted Testing) — vốn dựa trên việc xây dựng hàng trăm kịch bản kiểm thử (Test Cases) chi tiết từng bước (Step-by-step) — các dự án nhỏ sẽ ngay lập tức rơi vào bế tắc. Việc viết, cập nhật và duy trì một hệ thống kịch bản kiểm thử đồ sộ tiêu tốn quá nhiều tài nguyên, trong khi không gian đầu vào của phần mềm liên tục thay đổi. Hệ quả tất yếu là giai đoạn kiểm thử thường bị cắt xén một cách khiên cưỡng để kịp tiến độ release, dẫn đến việc sản phẩm được đưa đến tay khách hàng với đầy rẫy lỗi logic hệ thống nghiêm trọng.

Để giải quyết triệt để mâu thuẫn giữa "Nguồn lực giới hạn của dự án nhỏ" và "Yêu cầu chất lượng ngày càng cao của sản phẩm", trường phái Kiểm thử định hướng bối cảnh (Context-Driven Testing - CDT) nổi lên như một giải pháp cứu cánh mang tính chiến lược. Thay vì coi kiểm thử là một tập hợp các quy tắc cố định, đóng băng, CDT đặt bối cảnh thực tế của dự án lên hàng đầu và coi trọng tư duy phản biện, kỹ năng điều tra độc lập của người thực hiện. Nhận thấy tầm quan trọng của việc tối ưu hóa quy trình kiểm thử trong môi trường tài nguyên hạn hẹp, tác giả đã quyết định lựa chọn đề tài: "Ứng dụng kiểm thử định hướng bối cảnh (Context-Driven Testing) trong dự án phần mềm nhỏ" làm bài tiểu luận nghiên cứu của mình. Nghiên cứu này không chỉ dừng lại ở mức độ lý thuyết mà sẽ đi sâu vào việc vận dụng thực tế trên một hệ thống cụ thể, sử dụng các công cụ hiện đại và tinh gọn nhất nhằm chứng minh tính khả thi và hiệu quả của trường phái này.

1.2. Mục tiêu nghiên cứu

Đề tài tập trung nghiên cứu sâu sắc nhằm giải quyết ba mục tiêu cốt lõi sau:

- Về mặt lý thuyết học thuật: Tiến hành hệ thống hóa một cách toàn diện nền tảng lý luận về trường phái Kiểm thử định hướng bối cảnh (CDT). Phân tích chi tiết mối liên hệ biện chứng giữa các mô hình phân phối phần mềm (từ Waterfall đến Agile, DevOps) với chiến lược quản lý rủi ro chất lượng dựa trên tài liệu gốc của Heusser & Larsen (2024). Làm rõ 7 nguyên tắc nền tảng định hình nên tư duy CDT.
- Về mặt thực tiễn kỹ thuật: Thiết kế và hiện thực hóa một quy trình thực hành CDT tối ưu, loại bỏ hoàn toàn sự chồng chéo của Test Case truyền thống. Vận dụng trực tiếp lý thuyết vào một ngữ cảnh cụ thể: Hệ thống Quản lý máy tính (QuanLyPC) thông qua các công cụ trực quan hóa chiến lược (Mindmap) và công cụ thực thi kiểm thử tăng nghiệp vụ hiệu năng cao (Thunder Client) ngay trên môi trường lập trình Visual Studio Code (VS Code).

- Về mặt quản lý và đánh giá: Chứng minh năng lực tối ưu hóa chi phí và thời gian của giải pháp CDT thông qua các công cụ quản lý trực quan như Khung kiểm thử tinh gọn (Lean Test Canvas). Thiết lập một cơ chế phản hồi thông tin rủi ro nhanh chóng, chính xác (qua Session Log) nhằm hỗ trợ ban quản trị đưa ra quyết định phát hành phần mềm một cách an toàn.

1.3. Phạm vi nghiên cứu

Để đảm bảo tính tập trung cao độ và đi sâu vào bản chất của vấn đề theo đúng yêu cầu học thuật, phạm vi nghiên cứu của tiểu luận được giới hạn chặt chẽ như sau:

- Đối tượng nghiên cứu trọng tâm: Các nguyên lý cốt lõi của trường phái Kiểm thử định hướng bối cảnh (CDT); mô hình Khung kiểm thử tinh gọn (Lean Test Canvas); kỹ thuật xây dựng Bản tổng điều tra rủi ro (Census of Risk) và phương pháp quản lý phiên kiểm thử dựa trên thời gian (Session-based Test Management).
- Ngữ cảnh ứng dụng thực tế: Hệ thống phần mềm quản lý máy tính nội bộ (QuanLyPC). Hệ thống này được giả định phát triển dưới dạng một ứng dụng tinh gọn, giao tiếp dữ liệu chủ yếu qua các điểm cuối (Endpoints) của kiến trúc RESTful API, vận hành trong một đội ngũ lập trình nhỏ (từ 3-5 người) và không có nhân sự QA chuyên trách độc lập.
- Môi trường công nghệ và công cụ thực hành: Toàn bộ hoạt động thực hành kỹ thuật được thực hiện trực tiếp trên trình soạn thảo mã nguồn Visual Studio Code (VS Code). Trong đó, tác giả sử dụng hai tiện ích mở rộng cốt lõi bao gồm: vscode-mindmap (phát triển bởi Souche) để thiết kế cây sơ đồ rủi ro (QuanLyPC.km), và Thunder Client để viết, thực thi các kịch bản kiểm tra trạng thái phản hồi của API hệ thống. Toàn bộ nhật ký kiểm thử được chuẩn hóa bằng ngôn ngữ đánh dấu Markdown (CDT_Test_Log.md).

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ TỔNG QUAN

2.1. Phân tích các mô hình phân phối phần mềm và sự chuyển dịch của hoạt động kiểm thử

Theo nghiên cứu của Heusser và Larsen (2024) trong Chương 8, chiến lược kiểm thử của một tổ chức không bao giờ tồn tại biệt lập, mà nó luôn bị quy định một cách nghiêm ngặt bởi mô hình phân phối (Delivery Models) mà tổ chức đó đang áp dụng. Để hiểu rõ tại sao Kiểm thử định hướng bối cảnh là tất yếu, ta cần phân tích lịch sử dịch chuyển của các mô hình này:

2.1.1. Mô hình Thác nước (Waterfall) và V-Model truyền thống

Trong mô hình Thác nước truyền thống, các giai đoạn phát triển được thực hiện một cách tuần tự và tách biệt tuyệt đối: Thu thập yêu cầu, Thiết kế kiến trúc, Viết mã nguồn, và cuối cùng mới đến giai đoạn Kiểm thử (Testing). Heusser và Larsen (2024) chỉ ra rằng, trong bối cảnh này, kiểm thử vô tình biến thành "chốt chặn cuối cùng" và là nơi đầu tiên cung cấp cho tổ chức một cái nhìn khách quan, thực tế về việc liệu phần mềm có thực sự hoạt động hay không.

V-Model ra đời như một sự cải tiến nhằm gắn kết các pha kiểm thử tương ứng với các pha thiết kế (ví dụ: Unit Test đi kèm với Detail Design, System Test đi kèm với Requirements). Tuy nhiên, nhược điểm cốt lõi của cả hai mô hình này là việc tích hợp diễn ra quá muộn. Toàn bộ rủi ro kỹ thuật bị dồn nén lại ở cuối chu kỳ. Khi phát hiện ra các lỗi nghiêm trọng về mặt kiến trúc ở giai đoạn này, chi phí để sửa chữa và làm lại (Rework) là cực kỳ khủng khiếp, thường dẫn đến việc dự án bị vỡ tiến độ hoặc cạn kiệt ngân sách.

2.1.2. Mô hình Agile (Scrum và Extreme Programming - XP)

Sự ra đời của Tuyên ngôn Agile đã làm đảo lộn cấu trúc tuần tự của Waterfall. Agile chia nhỏ chu kỳ phát triển thành các phân đoạn ngắn gọi là các Sprint (thường từ 1 đến 4 tuần). Ở đó, hoạt động phân tích, lập trình và kiểm thử được diễn ra đồng thời trong cùng một phân đoạn. Đặc biệt, mô hình Extreme Programming (XP) đẩy các thực hành kỹ thuật lên mức cực hạn thông qua Phát

triển định hướng kiểm thử (TDD - Test-Driven Development), lập trình cặp (Pair Programming) và tích hợp liên tục (Continuous Integration).

Tuy nhiên, Agile lại tạo ra một áp lực khổng lồ khác lên hoạt động QA: Áp lực hồi quy (Regression Pressure). Cứ sau mỗi Sprint, một lượng mã nguồn mới lại được đắp thêm vào hệ thống. Nếu không có một chiến lược thông minh, thời gian dành cho việc kiểm thử hồi quy (kiểm tra xem mã nguồn mới có làm hỏng tính năng cũ không) sẽ phình to theo cấp số nhân, chiếm trọn toàn bộ thời gian của Sprint và bóp nghẹt năng lực phát hành tính năng mới của đội ngũ.

2.1.3. Mô hình DevOps và Chuyển giao liên tục (Continuous Delivery)

DevOps và Continuous Delivery (CD) đại diện cho đỉnh cao hiện tại của mô hình phân phối phần mềm, nơi ranh giới giữa phát triển (Dev) và vận hành (Ops) hoàn toàn bị xóa nhòa. Mã nguồn sau khi được lập trình viên đẩy lên hệ thống quản lý phiên bản (Git) sẽ tự động kích hoạt đường ống CI/CD để tự động build, chạy Unit Test, triển khai lên môi trường Staging và đẩy thẳng lên môi trường vận hành (Production).

Trong bối cảnh CD chuyên sâu, các chu kỳ kiểm thử hồi quy thủ công kéo dài hàng tuần hoàn toàn bị khai tử. Thay vào đó, tổ chức sử dụng các kỹ thuật tiên tiến như Kiến trúc vi dịch vụ (Microservices), Cơ chế cờ tính năng (Feature Flags) để bật/tắt tính năng linh hoạt, và các chiến lược triển khai an toàn như Blue/Green Deployment hoặc Canary Deployment. Kiểm thử lúc này không còn là một giai đoạn, mà trở thành một hoạt động được nhúng tự động và liên tục vào mọi điểm của đường ống phân phối.

2.2. Bản chất và 7 nguyên tắc cốt lõi của Trường phái Kiểm thử định hướng bối cảnh (CDT)

Được khởi xướng và phát triển bởi các chuyên gia huyền thoại như Cem Kaner, James Bach, Brian Marick và Brett Pettichord, trường phái Kiểm thử định hướng bối cảnh (CDT) ra đời như một lời phản kháng mạnh mẽ chống lại xu hướng rập khuôn, máy móc trong ngành QA. CDT tuyên bố rằng không có bất kỳ một quy trình, một biểu mẫu hay một công cụ nào là "vạn năng" cho mọi dự

án. Bản chất của CDT là đặt con người thực hiện kiểm thử vào vị trí trung tâm — một người điều tra độc lập, có tư duy phản biện, tự chủ và chịu trách nhiệm toàn diện về chiến lược của mình dựa trên thực trạng của dự án.

Trường phái tư duy này vận hành nghiêm ngặt dựa trên 7 nguyên tắc cốt lõi sau

STT	Nguyên tắc cốt lõi của trường phái CDT
1	Giá trị của bất kỳ thực hành nào đều phụ thuộc vào bối cảnh của nó
2	Có những thực hành tốt trong từng bối cảnh cụ thể, nhưng không có ‘Thực hành tốt nhất’.
3	Con người, Khi làm việc cùng nhau, là phần quan trọng nhất trong bối cảnh của bất kì dự án nào.
4	Các dự án tiến triển theo thời gian theo những cách thường không thể dự đoán trước.
5	Sản phẩm là 1 giải pháp. Nếu vấn đề không được giải quyết, sản phẩm đó coi như không hoạt động.
6	Kiểm thử phần mềm tốt là 1 quá trình tư duy trí tuệ đầy thách thức
7	Chỉ thông qua phán đoán và kĩ năng, thực thi hợp tác, chúng ta mới làm đúng việc vào đúng thời điểm.

Hình 2.2 : Bảng nguyên tắc cốt lõi của CDT

Nguyên tắc 1: Giá trị của bất kỳ thực hành nào đều phụ thuộc vào bối cảnh của nó (The value of any practice depends on its context). Một kỹ thuật kiểm thử tự động hóa giao diện cực kỳ thành công ở một tập đoàn lớn có thể trở thành một thảm họa gây lãng phí tài nguyên nếu áp dụng vào một startup công nghệ đang cần thay đổi tính năng hàng ngày.

Nguyên tắc 2: Có những thực hành tốt trong từng bối cảnh cụ thể, nhưng không có "thực hành tốt nhất" (There are good practices in context, but there are no "best practices"). Việc tuyên bố một phương pháp là "Best Practice" một cách vô điều kiện là biểu hiện của sự lười biếng trong tư duy. Mỗi dự án đòi hỏi một sự may đo chiến lược riêng biệt.

Nguyên tắc 3: Con người, khi làm việc cùng nhau, là phần quan trọng nhất trong bối cảnh của bất kỳ dự án nào (People, working together, are the most important part of any project's context). Công cụ hay quy trình chỉ là vật vô tri; chính kỹ năng, sự giao tiếp thông suốt và sự đồng lòng của đội ngũ mới là yếu tố quyết định chất lượng sản phẩm.

Nguyên tắc 4: Các dự án tiến triển theo thời gian theo những cách thường không thể dự đoán trước (Projects progress over time in ways that are often not predictable). Bản kế hoạch kiểm thử tĩnh được viết từ đầu dự án sẽ nhanh chóng trở thành phế tích. Người kiểm thử phải liên tục điều chỉnh chiến lược theo dòng chảy biến động của dự án.

Nguyên tắc 5: Sản phẩm là một giải pháp. Nếu vấn đề không được giải quyết, sản phẩm đó coi như không hoạt động (The product is a solution. If the problem isn't solved, the product doesn't work). Phần mềm có thể vượt qua 100% các dòng mã kiểm thử, nhưng nếu nó không giải quyết được nỗi đau thực tế của người dùng hoặc làm khách hàng thất vọng, đó vẫn là một sản phẩm thất bại về mặt chất lượng.

Nguyên tắc 6: Kiểm thử phần mềm tốt là một quá trình tư duy trí tuệ đầy thách thức (Good software testing is a challenging intellectual process). Kiểm thử không phải là hành động cơ học: đọc kịch bản, bấm chuột, điền kết quả Pass/Fail. Đó là một quá trình thám tử, đòi hỏi sự hoài nghi khoa học, tư duy phân tích logic và khả năng thiết kế thí nghiệm liên tục.

Nguyên tắc 7: Chỉ thông qua phán đoán và kỹ năng, được thực thi một cách hợp tác trong suốt dự án, chúng ta mới có thể làm đúng việc vào đúng thời điểm để kiểm thử sản phẩm một cách hiệu quả (Only through judgment and skill, exercised cooperatively throughout the project, can we do the right things at the right times to effectively test our products). Sự kết hợp giữa năng lực cá nhân và cơ chế cộng tác thời gian thực là chìa khóa để tối ưu hóa hiệu suất QA.

2.3. Nghiên cứu sâu về các kỹ thuật cụ thể trong Chương 10

2.3.1. Khung kiểm thử tinh gọn (Lean Test Canvas)

Để đưa tư duy CDT từ lý thuyết vào thực tế quản trị, Heusser (2014) đã phát triển một công cụ trực quan hóa mạnh mẽ mang tên Lean Test Canvas. Lấy cảm hứng từ Business Model Canvas và Toyota Way trong sản xuất tinh gọn, Lean Test Canvas cho phép tóm gọn toàn bộ trạng thái và chiến lược kiểm thử của một đội ngũ, một dự án trên một trang giấy duy nhất. Điều này giúp phát hiện ra ngay lập tức "con cá chết trên bàn" (dead fish on the table) — tức là những nút thắt cổ chai, những điểm yếu cốt lõi đang làm trì trệ quy trình phân phối.

Khung canvas này bao gồm các thành phần chiến lược quan trọng:

- Khách hàng (Customers): Định vị rõ ai là người tiêu thụ thông tin do hoạt động kiểm thử cung cấp (ví dụ: Bản thân đội ngũ phát triển, hay các nhà quản lý cấp cao).
- Tuyên bố giá trị (Value Proposition): Trả lời câu hỏi cốt lõi: *"Tại sao doanh nghiệp phải chi tiền cho hoạt động kiểm thử này?"* Hoạt động kiểm thử phải chứng minh được tỷ suất hoàn vốn (ROI), chứng minh \$1 chi cho QA sẽ bảo vệ được ít nhất \$1.25 giá trị kinh doanh.
- Đường ống kiểm thử và triển khai (Test and deploy pipeline): Đo lường chính xác khoảng thời gian mã nguồn di chuyển từ lúc hoàn thành đến khi lên Production, vạch trần các điểm nghẽn thời gian.
- Phạm vi cốt lõi (Core scope of role): Danh sách các loại rủi ro kỹ thuật mà đội ngũ QA phải chịu trách nhiệm (ví dụ: Chức năng, Hiệu năng API).
- Phần nằm ngoài phạm vi (Out of scope): Tuyên bố rõ ràng những gì đội ngũ không thực hiện (ví dụ: Bảo mật, Khả năng tiếp cận). Điều này giúp bảo vệ nguồn lực tối đa cho dự án nhỏ, tránh việc kỳ vọng mơ hồ từ các bên liên quan.

2.3.2. Sơ đồ tổng điều tra rủi ro (Census of Risk) thay thế cho Test Case

Một trong những đóng góp mang tính cách mạng của Chương 10 là việc thẳng thắn phê phán hệ thống tài liệu Test Case truyền thống. Heusser và Larsen (2024) khẳng định rằng việc viết Test Case chi tiết từng bước tạo ra những khối tài liệu khổng lồ trông thì có vẻ quan trọng nhưng thực chất lại vô giá trị, làm giảm quyền tự quyết và triệt tiêu năng lực bao phủ kiểm thử theo thời gian vì bất người test lặp đi lặp lại một lỗi mòn cũ kỹ.

CDT thay thế bằng Bản tổng điều tra rủi ro (Census of Risk) — một danh sách cuốn chiếu (rolling list) liên tục ghi nhận tất cả những gì có thể đi chệch hướng trong hệ thống. Thay vì viết các bước dài dòng, Census of Risk chỉ lưu trữ các Ý tưởng kiểm thử (Test Ideas) dưới dạng các tiêu đề ngắn gọn (ví dụ: *Path to Purchase* hoặc *API Timeout*). Người kiểm thử sẽ kết hợp các ý tưởng này với một cuốn "sách công thức" (Recipe book) lưu trữ các thao tác kỹ thuật cơ bản và tiến hành thực thi kiểm thử khám phá (Exploratory Testing) trong các khung thời gian đóng gói cố định (Timebox, thường là 30-45 phút). Ban quản lý sẽ trực tiếp tham gia vào việc đặt độ ưu tiên (Priority) cho danh sách rủi ro này, từ đó tự tay vạch ra đường giới hạn cắt giảm (Cut Line) — nghĩa là chấp nhận những rủi ro có độ ưu tiên thấp bị bỏ lại phía dưới nếu hết thời gian và ngân sách.

2.3.3. Phương pháp tiếp cận trực quan hóa qua Bản đồ tư duy (Mindmap Regression)

Đối với các dự án có giao diện và logic thay đổi liên tục, việc sử dụng Sơ đồ tư duy (Mindmap) để theo dõi kiểm thử hồi quy là một thực hành CDT cực kỳ hiệu quả. Toàn bộ hệ thống được phân rã thành một cây tính năng phân cấp. Khi thực hiện kiểm thử, người kiểm thử sẽ tiến hành "tô màu" trực tiếp lên các nút (Nodes) của sơ đồ tư duy: màu trắng cho phần chưa chạy, màu xanh cho tính năng ổn định, màu đỏ/vàng cho các phần có lỗi, và màu xám cho các phần bị bỏ qua (Skipped). Bất kỳ ai, từ lập trình viên đến giám đốc điều hành, chỉ cần nhìn vào bức tranh Mindmap là ngay lập tức nắm bắt được trạng thái chất lượng của phần mềm mà không cần đọc qua các bảng báo cáo Excel dày đặc thông số.

CHƯƠNG 3: VẬN DỤNG VÀ THỰC HÀNH TẠI NGŨ CẢNH CỤ THỂ

3.1. Mô tả chi tiết bài toán và ngữ cảnh thực tế của hệ thống Quản lý PC

3.1.1. Đặt vấn đề và kiến trúc hệ thống

Hệ thống Quản lý PC (QuanLyPC) là một ứng dụng phần mềm nội bộ được thiết kế nhằm mục đích tối ưu hóa quy trình theo dõi, cấu hình, cấp phát và bảo trì hạ tầng máy tính tại các cơ quan, tổ chức doanh nghiệp. Hệ thống này có nhiệm vụ quản lý vòng đời của một thiết bị phần cứng, từ lúc nhập kho, phân phối cho nhân sự, bảo trì định kỳ cho đến khi thanh lý.

Về mặt kiến trúc kỹ thuật, hệ thống được xây dựng theo mô hình Single Page Application (SPA) ở tầng giao diện kết hợp với kiến trúc RESTful API ở tầng mã nguồn phía sau (Backend). Toàn bộ các tương tác nghiệp vụ, dữ liệu cấu hình chi tiết của máy tính (như CPU, RAM, ổ cứng, địa chỉ MAC, hệ điều hành) và trạng thái hoạt động (Active, Maintenance, Disposed) đều được đóng gói và truyền tải dưới dạng dữ liệu định dạng JSON (JavaScript Object Notation) qua các điểm cuối (Endpoints) của API.

3.1.2. Phân tích bối cảnh dự án (Project Context Analysis) theo mô hình CDT

Để áp dụng Kiểm thử định hướng bối cảnh (CDT), việc đầu tiên và quan trọng nhất là phải phân tích tường tận các yếu tố bối cảnh ràng buộc xung quanh dự án Quản lý PC, thay vì nhảy vào viết kịch bản kiểm thử một cách mù quáng. Bối cảnh thực tế của dự án bao gồm 4 trục xoay cốt lõi:

- **Rủi ro kỹ thuật (Risk-driven):** Do hệ thống phải xử lý lượng dữ liệu cấu hình phần cứng rất chi tiết, rủi ro lớn nhất nằm ở việc sai lệch logic dữ liệu khi có nhiều yêu cầu cập nhật (Update) trạng thái máy tính cùng một lúc. Ngoài ra, việc rò rỉ mã token xác thực người dùng trên các API Endpoint truy xuất thông tin thiết bị nhạy cảm là một nguy cơ có độ nghiêm trọng cao cần được rà soát kỹ lưỡng.

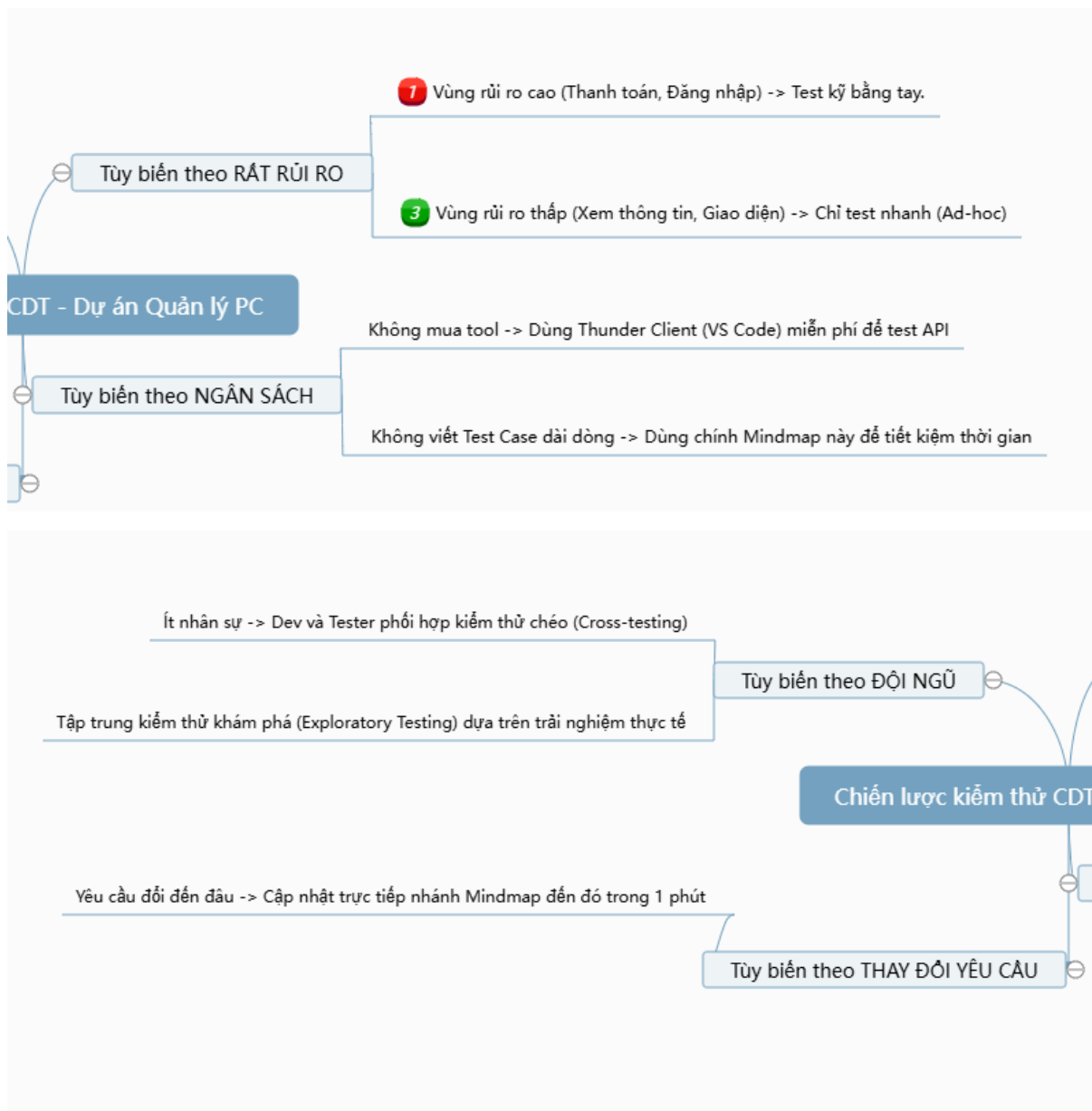
- Ngân sách hạ tầng (Budget-driven): Đây là một dự án phần mềm quy mô nhỏ, phục vụ nội bộ, do đó ngân sách được cấp phát ở mức cực kỳ tối giản. Dự án không có chi phí dành cho các hệ thống quản lý lỗi thương mại phức tạp, không có hạ tầng máy chủ lớn để thiết lập các môi trường giả lập (Mocking Environment) đắt đỏ, và không có ngân sách mua bản quyền cho các công cụ kiểm thử tự động hóa giao diện (UI Automation Tools) công kênh.
- Đội ngũ nhân sự (Team-driven): Đội ngũ phát triển dự án có quy mô siêu nhỏ (chỉ từ 3 đến 5 lập trình viên). Dự án hoàn toàn không có nhân sự QA/Tester chuyên trách. Vai trò đảm bảo chất lượng được chia sẻ trực tiếp cho các kỹ sư lập trình. Do đó, quy trình kiểm thử đòi hỏi phải cực kỳ nhanh, tích hợp thẳng vào không gian làm việc của lập trình viên để tránh lãng phí thời gian bàn giao hoặc viết tài liệu rườm rà.
- Mức độ thay đổi yêu cầu (Volatility-driven): Đặc tả các dòng máy tính, linh kiện phần cứng và quy trình phê duyệt cấp phát máy tính của tổ chức liên tục thay đổi theo các quý báu và chiến lược vận hành. Điều này dẫn đến cấu trúc cơ sở dữ liệu và tài liệu thiết kế API của hệ thống Quản lý PC bị biến động liên tục. Nếu sử dụng Test Case truyền thống, đội ngũ sẽ mất toàn bộ thời gian chỉ để sửa tài liệu mỗi khi API thay đổi.

3.2. Hiện thực hóa giải pháp kiểm thử CDT tinh gọn trên Visual Studio Code (VS Code)

Toàn bộ quy trình thực hành kỹ thuật của đề tài được triển khai khép kín ngay trong trình soạn thảo mã nguồn **VS Code**, tận dụng tối đa hệ sinh thái Extension miễn phí để tạo ra một không gian làm việc hiệu năng cao, phản hồi nhanh cho lập trình viên kiêm kiểm thử viên.

Bước 1: Trực quan hóa bản đồ tổng điều tra rủi ro bằng vscode-mindmap

Thay vì soạn thảo file Excel dày đặc, tác giả khởi tạo file sơ đồ tư duy QuanLyPC.km thông qua tiện ích mở rộng vscode-mindmap (tác giả Souche). Cấu trúc cây sơ đồ tư duy được thiết lập trực quan với nút gốc là "Chiến lược CDT Quản lý PC", từ đó phân rã thành 4 nhánh bối cảnh lớn tương ứng với các phân tích ở mục 3.1.2. Dưới đây là cấu trúc biểu diễn dưới dạng cây thư mục của file sơ đồ tư duy rủi ro:



Hình 3.2 : Sơ đồ rủi ro

Bước 2: Thiết lập kịch bản mã nguồn và thực thi kiểm thử API bằng Thunder Client

Để minh chứng cho tính thực tiễn và năng lực phát hiện lỗi sớm ở tầng dưới (tầng nghiệp vụ API), tác giả tiến hành thực thi một yêu cầu kiểm thử API Endpoints. Tiện ích Thunder Client trong VS Code được lựa chọn nhờ ưu điểm khởi động cực nhanh (dưới 1 giây), chiếm ít bộ nhớ RAM hơn rất nhiều so với công cụ Postman truyền thống, giúp máy tính phát triển của lập trình viên luôn duy trì hiệu năng tối ưu.

- Đoạn mã cấu trúc dữ liệu JSON phản hồi giả lập của hệ thống (Endpoint `/posts/1` đại diện cho thông tin PC ID số 1):

```
{  
  "userId": 1,  
  "id": 1,  
  "title": "Cấu hình PC văn phòng ban Kế toán - Mã PC: 58KTP-01",  
  "body": "Thông tin chi tiết: CPU Intel Core i5, 16GB RAM, 512GB SSD. Trạng thái: Đang hoạt động bình thường trên hệ điều hành Ubuntu Linux."  
}
```

3.3. Phân tích kết quả, thảo luận và đánh giá hiệu quả toàn diện

3.3.1. Đánh giá định lượng về mặt thời gian và chi phí vận hành

Quá trình vận dụng thực tế phương pháp luận CDT vào hệ thống Quản lý PC đã mang lại những cải tiến mang tính đột phá về mặt hiệu suất hoạt động cho đội ngũ dự án nhỏ. Khi tiến hành so sánh định lượng với quy trình kiểm thử truyền thống (Scripted Testing dựa trên việc viết kịch bản Excel chi tiết), kết quả ghi nhận như sau:

- Thời gian thiết lập và chuẩn bị tài liệu: Quy trình truyền thống yêu cầu kỹ sư phải ngồi viết chi tiết từng bước (Bước 1: Truy cập link, Bước 2: Nhập thông tin, Bước 3: Nhấn nút, Kết quả mong đợi...). Việc này tiêu tốn từ 16 đến 24 giờ làm việc cho một phân hệ tính năng. Với CDT, việc phác thảo cây rủi ro trên Mindmap và danh sách ý tưởng trong

Census of Risk chỉ tiêu tốn đúng 1 giờ làm việc, tiết kiệm hơn 90% thời gian chuẩn bị tài liệu.

- Thời gian phát hiện lỗi và phản hồi (Feedback Loop): Thay vì phải chờ đợi toàn bộ giao diện người dùng (UI) hoàn thiện vào cuối chu kỳ phát triển mới có thể tiến hành test tổng thể, công cụ Thunder Client cho phép lập trình viên kiểm tra độ ổn định của API ngay khi vừa viết xong dòng mã nguồn cuối cùng. Chu kỳ phản hồi thông tin lỗi được rút ngắn từ hàng tuần xuống còn vài giây.

3.3.2. Bảng tổng hợp so sánh các chỉ số chất lượng cốt lõi

Để làm nổi bật tính khả thi của đề tài, bảng dưới đây tổng hợp toàn diện sự khác biệt trước và sau khi áp dụng mô hình chiến lược cải tiến của CDT vào dự án:

Chỉ số / Tiêu chí đánh giá	Cách tiếp cận truyền thống (Scripted Testing)	Giải pháp cải tiến áp dụng CDT
Độ phủ kiểm thử (Test Coverage)	Bị giới hạn nghiêm ngặt trong phạm vi các dòng chữ đã được viết sẵn trong kịch bản Test Case.	Không giới hạn. Người kiểm thử liên tục mở rộng không gian test dựa trên tư duy khám phá và biến động thực tế.
Công cụ và Hạ tầng quản lý	Phức tạp, công kênh, đòi hỏi cài đặt phần mềm bên thứ ba độc lập, tốn tài nguyên máy tính.	Tinh gọn, tích hợp 100% bên trong IDE VS Code thông qua các extension mã nguồn mở, hoàn toàn miễn phí.
Khả năng bảo trì tài liệu	Cực kỳ thấp. Mỗi khi cấu hình API thay đổi, toàn bộ hệ thống file	Cực kỳ cao. Chỉ cần cập nhật, thêm hoặc xóa các nút (Nodes) trực tiếp

	Excel Test Case phải được viết lại từ đầu.	trên file sơ đồ tư duy QuanLyPC.km.
Tính chủ động của nhân sự	Người test hoạt động như một cỗ máy, lặp đi lặp lại các bước có sẵn, triệt tiêu khả năng phát hiện lỗi logic sâu.	Người test đóng vai trò một thám tử, liên tục tư duy phản biện, thiết lập các thí nghiệm để săn tìm các lỗi biên phức tạp.
Cơ chế ra quyết định phát hành	Dựa vào tỷ lệ phần trăm số lượng Test Case đạt (Pass Rate) một cách máy móc, dễ tạo ra cảm giác an toàn giả tạo.	Dựa vào đường giới hạn "Cut Line" trên Bản tổng điều tra rủi ro, giúp ban quản lý nắm chắc những mối nguy cơ thực tế.

Hình 3.3.2 Hình so sánh các chỉ số chất lượng

Kết luận thảo luận: Thực tiễn vận dụng cho thấy, trường phái CDT không hề làm giảm đi chất lượng của sản phẩm phần mềm, mà ngược lại, nó giúp tối ưu hóa một cách thông minh nguồn tài nguyên khan hiếm của dự án nhỏ. Bằng cách loại bỏ các thủ tục hành chính giấy tờ không cần thiết, CDT tập trung 100% năng lượng của đội ngũ vào những vùng tính năng có rủi ro cao nhất, đảm bảo hệ thống Quản lý PC luôn đạt trạng thái chất lượng "Đủ tốt để phát hành" (Good-enough quality) trong một khung thời gian và ngân sách nghiêm ngặt nhất.

CHƯƠNG 4: KẾT LUẬN VÀ KIẾN NGHỊ

4.1. Tổng kết nội dung và các phát hiện chính

Qua quá trình nghiên cứu lý thuyết nền tảng từ công trình của Heusser và Larsen (2024) kết hợp với việc triển khai vận dụng thực hành kỹ thuật trực tiếp trên hệ thống Quản lý PC, đề tài nghiên cứu về trường phái Kiểm thử định hướng bối cảnh (CDT) đã đạt được những kết quả mang tính bước ngoặt sau:

- Về mặt nhận thức lý luận: Bài tiểu luận đã chứng minh một cách khoa học rằng chiến lược đảm bảo chất lượng phần mềm không bao giờ là một hằng số cố định. Chất lượng của hoạt động QA chỉ thực sự được tối ưu khi nó được đặt trong mối tương quan biện chứng với mô hình phân phối và các ràng buộc thực tế của tổ chức. Bảy nguyên tắc cốt lõi của CDT không chỉ là những khẩu hiệu lý thuyết, mà chúng là kim chỉ nam giúp giải phóng tư duy của kỹ sư phần mềm khỏi những biểu mẫu Test Case đóng băng, lỗi thời của kỷ nguyên cũ.
- Về mặt thực tiễn kỹ thuật: Đề tài đã hiện thực hóa thành công một khung làm việc (Testing Framework) siêu tinh gọn ngay trong môi trường phát triển Visual Studio Code (VS Code). Việc thay thế tài liệu kiểm thử truyền thống bằng file sơ đồ tư duy rủi ro QuanLyPC.km (vscode-mindmap) và sử dụng Thunder Client để kiểm thử tự động hóa tầng API đã tạo ra một vòng phản hồi thông tin lỗi (Feedback Loop) tính bằng giây. Điều này chứng minh rằng các dự án nhỏ hoàn toàn có thể kiểm soát chất lượng một cách toàn diện mà không cần tiêu tốn ngân sách cho các công cụ thương mại đắt đỏ.
- Về mặt quản trị rủi ro: Bằng việc ứng dụng mô hình Khung kiểm thử tinh gọn (Lean Test Canvas) và Bản tổng điều tra rủi ro (Census of Risk), dự án đã thiết lập được một cơ chế vạch đường giới hạn "Cut Line" vô cùng minh bạch. Nhà quản lý dự án giờ đây có thể nhìn thấy rõ ràng những mối nguy cơ nào đã được giảm thiểu, những rủi ro nào được chấp nhận bỏ lại phía sau dựa trên bài toán kinh doanh và áp lực

thời gian thực tế, thay vì rơi vào trạng thái an tâm giả tạo của các báo cáo số lượng thông thường.

4.2. Hạn chế mang tính khách quan của giải pháp

Mặc dù phương pháp luận CDT mang lại những cải tiến đột phá cho các dự án quy mô nhỏ, tác giả vẫn cần nhìn nhận một cách khách quan các giới hạn nội tại của giải pháp khi đưa vào vận hành thực tế:

- Sự phụ thuộc nghiêm ngặt vào yếu tố con người: Do CDT loại bỏ các tài liệu kịch bản từng bước (Step-by-step), quy trình này đòi hỏi người kiểm thử (hoặc lập trình viên kiêm nhiệm) phải có tư duy phản biện sắc bén, sự hoài nghi khoa học và tính tự giác cực kỳ cao. Nếu nhân sự có năng lực kỹ thuật yếu hoặc thiếu trách nhiệm, Bản tổng điều tra rủi ro (Census of Risk) sẽ nhanh chóng bị bỏ sót và biến thành một danh sách kiểm tra hời hợt.
- Thử thách về mặt quy mô khi hệ thống phình to: Mô hình quản lý trực quan qua sơ đồ tư duy và ghi log dạng file Markdown (CDT_Test_Log.md) hoạt động hoàn hảo trong phạm vi một đội ngũ nhỏ từ 3 đến 5 người với một attack surface (bề mặt tấn công/rủi ro) giới hạn. Khi hệ thống Quản lý PC mở rộng lên quy mô lớn với hàng trăm phân hệ API giao tiếp chéo, nếu không có một chiến lược phân rã kiến trúc tốt, cây sơ đồ tư duy sẽ trở nên quá tải, rối rắm và cực kỳ khó duy trì cấu trúc đồng bộ.

4.3. Định hướng mở rộng và Kiến nghị trong tương lai

Để phát huy tối đa giá trị của công trình nghiên cứu và giúp hệ thống thích ứng tốt hơn khi tổ chức tăng trưởng, tác giả đề xuất các định hướng cải tiến sau:

- Tích hợp sâu vào đường ống tự động hóa (CI/CD Pipeline): Trong các giai đoạn tiếp theo, các kịch bản kiểm thử API được viết trên Thunder Client cần được xuất ra định dạng mã nguồn tiêu chuẩn và nhúng trực tiếp vào chu kỳ chạy tự động của Git Actions hoặc Jenkins. Điều này

giúp biến hoạt động kiểm thử API từ thủ công (Manual API testing) thành các bài kiểm tra giao ước tự động (Contract Tests) chạy liên tục trên môi trường Staging trước khi thực hiện deploy.

- Dịch chuyển sang mô hình Kỹ thuật độ tin cậy trang web (SRE): Khi phần mềm được đưa lên môi trường Production, đội ngũ cần ứng dụng các thực hành của DevOps và SRE (Site Reliability Engineering) như đã được gợi ý trong Chương 8. Bằng cách thiết lập các hệ thống thu thập log tự động và biểu đồ giám sát hiệu năng thời gian thực (Real-time Monitoring Dashboards), người kiểm thử có thể thu thập thông tin rủi ro dựa trên hành vi thực tế của người dùng, từ đó liên tục làm giàu cho bản sơ đồ tư duy rủi ro ban đầu.
- Xây dựng "Sách công thức kỹ thuật" (Recipe Book): Đội ngũ phát triển cần đóng gói các thao tác kỹ thuật lặp đi lặp lại (như cách khởi tạo token xác thực, cách cấu hình database SQL Anywhere, cách bắt gói tin) thành một thư viện công thức nội bộ dạng Wiki. Việc này giúp giảm thiểu tối đa thời gian làm quen hệ thống cho các nhân sự mới và chuẩn hóa năng lực thực thi kiểm thử khám phá cho toàn đội ngũ.

TÀI LIỆU THAM KHẢO

- Heusser, M., & Larsen, M. (2024). *Software Testing Strategies: A context-driven approach to automation, planning, and testing* (1st ed.). Packt Publishing. [Trích dẫn trọng tâm: Chương 8 (Delivery Models and Testing, pp. 169–191) & Chương 10 (Putting Your Test Strategy Together, pp. 213–228)].
- Kaner, C., Bach, J., & Pettichord, B. (2001). *Lessons Learned in Software Testing: A Context-Driven Approach*. John Wiley & Sons.
- Heusser, M. (2014). *The Lean Test Canvas Heuristic*. Excelon Development. Retrieved June 2026, from <https://xndev.com/2014/11/the-lean-test-canvas/>.
- Royce, W. W. (1970). Managing the development of large software systems. *Proceedings of IEEE WESCON*, 26(8), 1–9.
- Schwaber, K. (1995). Scrum development process. *Proceedings of the 10th Annual ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages, and Applications (OOPSLA)*, 117–134.
- Agile Alliance. (2001). *Manifesto for Agile Software Development*. Retrieved from <https://agilemanifesto.org>.
- Johnson, K. N. (2009). *RCRCRC: A Heuristic for Regression Testing*. Retrieved from <http://karennicolejohnson.com/2009/11/a-heuristic-for-regression-testing/>.
- DeGrace, P., & Stahl, L. H. (1993). *Wicked Problems, Righteous Solutions: A Catalogue of Modern Software Engineering Paradigms*. Yourdon Press.

Kiểm thử định hướng bối cảnh trong PC

Gamma

Presenter

TABLE OF CONTENTS

- | | | | |
|-----------|-------------------|-----------|--------------------|
| 01 | Giới thiệu dự án | 04 | Nguyên tắc CDT |
| 02 | Lý do chọn đề tài | 05 | Thực hành kiểm thử |
| 03 | Cơ sở lý thuyết | 06 | Catalogue6 |





01 Giới thiệu dự án

Thông tin sinh viên và dự án giới thiệu

Thông tin sinh viên Nguyễn Tuấn Anh

- Sinh viên Nguyễn Tuấn Anh đang thực hiện dự án nghiên cứu về kiểm thử định hướng bối cảnh trong quản lý phần mềm, nhằm nâng cao hiệu quả kiểm thử và đảm bảo chất lượng phần mềm trong các dự án nhỏ.

Mục tiêu dự án và phạm vi nghiên cứu

- Dự án nhằm áp dụng phương pháp CDT vào quản lý dự án phần mềm, tập trung vào các nguyên tắc và công cụ hỗ trợ để nâng cao hiệu suất kiểm thử và giảm thiểu rủi ro.



Nền tảng lập trình và công nghệ sử dụng trong dự án

Mô hình phát triển phần mềm hiện đại

- Dự án dựa trên các mô hình phát triển phần mềm như Waterfall, Agile và DevOps để phù hợp với các quy trình quản lý linh hoạt và liên tục tích hợp.

Phương pháp kiểm thử và phân tích

- Áp dụng kiểm thử API, kiểm thử rủi ro, lập bản đồ tư duy và phân tích logs để phát hiện lỗi nhanh chóng, chính xác và giảm thiểu thời gian kiểm thử.



Các công cụ kiểm thử và CI/CD

- Sử dụng các công cụ như Thunder Client cho kiểm thử API, vscode-mindmap cho lập bản đồ rủi ro, Markdown để ghi nhật ký, nhằm tối ưu hóa quá trình kiểm thử và tự động hóa.

Kiến trúc hệ thống của ứng dụng quản lý PC

- Ứng dụng sử dụng kiến trúc SPA, API RESTful, JSON để đảm bảo khả năng mở rộng, dễ bảo trì và tích hợp trong môi trường phát triển hiện đại.

Thẻ loại dự án và ứng dụng thực tiễn

Dự án kiểm thử phần mềm hướng bối cảnh

- Dự án tập trung vào kiểm thử dựa trên ngữ cảnh của hệ thống, giúp các nhóm phát triển phần mềm nhỏ đảm bảo chất lượng mà không cần kiểm thử toàn diện phức tạp.

Mục tiêu nghiên cứu và kết quả đạt được



Nâng cao hiệu quả kiểm thử và giảm thiểu thời gian

- Thông qua việc áp dụng các nguyên tắc CDT, dự án đã giúp tiết kiệm tới 90% thời gian ghi chép tài liệu, tối ưu quy trình kiểm thử.



Xây dựng nền tảng tích hợp CI/CD cho dự án nhỏ

- Dự án hướng tới tích hợp liên tục và tự động hóa quy trình kiểm thử, từ đó nâng cao chất lượng phần mềm và khả năng mở rộng trong tương lai.



02 Lý do chọn đề tài

Giới hạn dự án nhỏ trong thực tế phát triển phần mềm

Các dự án nhỏ gặp giới hạn về nguồn lực và thời gian

- Các dự án nhỏ thường có hạn chế về nhân lực, ngân sách và thời gian, khiến cho việc kiểm thử truyền thống trở nên không khả thi hoặc gây chậm trễ lớn trong quá trình phát triển phần mềm.

Yêu cầu nhanh chóng và linh hoạt trong kiểm thử

- Trong môi trường nhỏ, yêu cầu kiểm thử nhanh, linh hoạt và phù hợp với sự thay đổi liên tục của dự án, đòi hỏi phương pháp kiểm thử phù hợp và hiệu quả hơn.



So với kiểm thử truyền thống, kiểm thử định hướng bối cảnh mang lại lợi ích gì



Tối ưu hóa quá trình kiểm thử phù hợp với từng bối cảnh

- Kiểm thử định hướng bối cảnh giúp tập trung vào các yếu tố quan trọng, phù hợp với đặc thù của dự án, từ đó nâng cao hiệu quả và giảm thiểu lãng phí thời gian.



Tăng khả năng thích ứng với thay đổi nhanh chóng

- Phương pháp này cho phép linh hoạt điều chỉnh kiểm thử dựa trên các yếu tố như môi trường, yêu cầu khách hàng, và tiến trình phát triển, khác biệt rõ rệt so với kiểm thử truyền thống cứng nhắc.



Thách thức liên quan đến tài nguyên trong kiểm thử phần mềm nhỏ

Hạn chế về nhân lực và công cụ kiểm thử

- Dự án nhỏ thường thiếu nhân lực chuyên môn cao và các công cụ tự động hóa, gây khó khăn trong việc thực hiện kiểm thử một cách toàn diện và hiệu quả.

Áp lực thời gian và ngân sách hạn hẹp

- Ngân sách hạn chế và thời gian gấp rút khiến cho việc thực hiện kiểm thử sâu rộng trở nên khó khăn, cần các giải pháp tối ưu để đảm bảo chất lượng.



Cần thiết có các giải pháp mới phù hợp với đặc thù dự án nhỏ



Áp dụng kiểm thử nhẹ, linh hoạt và thích ứng nhanh

- Giải pháp mới cần tập trung vào kiểm thử nhẹ, nhanh và phù hợp với môi trường nhỏ, giúp tiết kiệm thời gian và nguồn lực mà vẫn đảm bảo chất lượng phần mềm.



03 Cơ sở lý thuyết



Lý do chọn đề tài: So sánh giữa các hạn chế dự án nhỏ và kiểm thử truyền thống nặng nề



Nhu cầu kiểm thử linh hoạt trong dự án nhỏ

- Trong các dự án nhỏ, yêu cầu kiểm thử nhanh chóng và linh hoạt hơn so với các phương pháp truyền thống. Do đó, CDT phù hợp hơn để thích ứng với môi trường thay đổi liên tục.



Hạn chế của kiểm thử truyền thống

- Kiểm thử truyền thống thường chậm, cồng kềnh và không linh hoạt, gây ra khó khăn trong việc đáp ứng nhanh các yêu cầu thay đổi của dự án nhỏ.



Cơ sở lý thuyết về mô hình phát triển phần mềm: chuyển đổi từ Waterfall sang Agile và DevOps

Mô hình Waterfall truyền thống

- Mô hình Waterfall theo trình tự, cứng nhắc, dễ gây trì hoãn và khó thích ứng với các yêu cầu thay đổi trong quá trình phát triển phần mềm.

Chuyển sang mô hình Agile và DevOps

- Agile và DevOps thúc đẩy tính linh hoạt, liên tục hợp tác, tự động hóa và phản hồi nhanh chóng, phù hợp với những yêu cầu hiện đại về phát triển nhanh và chất lượng cao.



Nguyên tắc cốt lõi của Kiểm thử định hướng bối cảnh (CDT)



Tập trung vào bối cảnh cụ thể

- CDT nhấn mạnh việc hiểu rõ ngữ cảnh của dự án và điều chỉnh kiểm thử phù hợp để nâng cao hiệu quả và phù hợp với tình huống thực tế.



Tối ưu hóa các hoạt động kiểm thử

- Dựa vào kiến thức và kinh nghiệm, CDT hướng tới tập trung kiểm thử các lĩnh vực cần thiết, tránh lãng phí tài nguyên vào các hoạt động không phù hợp.



Mô hình thử nghiệm Lean Test Canvas trong kiểm thử phần mềm

Bảng ma trận kiểm thử nhẹ

- Lean Test Canvas giúp tổ chức các hoạt động kiểm thử một cách đơn giản, rõ ràng, tối ưu hóa nguồn lực và tăng khả năng phản hồi nhanh trong quá trình kiểm thử.



04 Nguyên tắc CDT

Nguyên tắc 1: Hiểu rõ ngữ cảnh dự án và yêu cầu khách hàng

Xác định rõ mục tiêu của dự án kiểm thử

- Hiểu rõ mục đích và phạm vi của dự án giúp xác định phương pháp phù hợp, đảm bảo chất lượng và tối ưu nguồn lực trong kiểm thử.

Phân tích đặc điểm của hệ thống quản lý PC

- Phân tích kiến trúc SPA, API RESTful và dữ liệu JSON để thiết kế kiểm thử phù hợp, tăng tính hiệu quả và chính xác.



Nguyên tắc 2: Áp dụng phương pháp kiểm thử phù hợp với ngữ cảnh



Chọn mô hình phát triển phù hợp (Waterfall, Agile, DevOps)

- Lựa chọn mô hình phù hợp giúp tối ưu quy trình kiểm thử, giảm thiểu rủi ro và nâng cao hiệu quả dự án.

Nguyên tắc 3: Tính linh hoạt trong chiến lược kiểm thử

Sử dụng Lean Test Canvas để tối giản quy trình

- Áp dụng sơ đồ kiểm thử nhẹ giúp linh hoạt điều chỉnh, phản ứng nhanh với các thay đổi của dự án và giảm thiểu lãng phí.

Nguyên tắc 4: Đánh giá hiệu quả và tối ưu hóa quy trình kiểm thử



Sử dụng các công cụ đo lường như biểu đồ phân tích

- So sánh thời gian và kết quả công việc để đánh giá hiệu quả, từ đó điều chỉnh quy trình phù hợp, nâng cao năng suất.



05 Thực hành kiểm thử

400 100

Giới thiệu về ứng dụng kiểm thử định hướng bối cảnh (CDT) trong dự án Quản lý PC

Khái niệm và tầm quan trọng của CDT trong kiểm thử phần mềm

- Phương pháp kiểm thử định hướng bối cảnh giúp tập trung vào ngữ cảnh thực tế, nâng cao hiệu quả và phù hợp với từng dự án cụ thể trong quản lý PC.

400 100

Cơ sở lý thuyết và các mô hình phát triển phần mềm liên quan

Mô hình phát triển phần mềm thay đổi từ Waterfall đến DevOps

- Chuyển đổi từ mô hình truyền thống Waterfall sang Agile và DevOps giúp rút ngắn chu kỳ phát triển, tăng tính linh hoạt và liên tục tích hợp kiểm thử.

Các nguyên tắc cốt lõi của kiểm thử định hướng bối cảnh (CDT)



7 nguyên tắc chính của CDT trong kiểm thử phần mềm

- Các nguyên tắc này nhấn mạnh tập trung vào ngữ cảnh, linh hoạt và thích ứng với các yếu tố thực tế để nâng cao hiệu quả kiểm thử.

Mô hình kiểm thử nhẹ nhàng và công cụ hỗ trợ

Lean Test Canvas và ứng dụng trong kiểm thử phần mềm

- Mô hình giúp tối giản quy trình kiểm thử, tập trung vào các yếu tố cốt lõi, dễ dàng lập kế hoạch và đánh giá.

**Thank you for
watching**

Gamma

Presenter

